Earned Autonomy: A Reversibility and Evidence Gate for Automated Infrastructure Remediation

Aman Das
Earned Autonomy Project
Remote, India

***Abstract*—Automated remediation can reduce the time required to recover from infrastructure failures, but allowing an automated system to act without sufficient evidence can introduce additional failures or increase the impact of an incorrect decision. This paper presents EarnedAutonomy, a trust gate for self-healing infrastructure that determines whether a candidate remediation action should be executed automatically, proposed for human approval, or escalated to a human operator. The gate evaluates three factors: action reversibility, blast radius, and accumulated evidence of previous outcomes. Evidence is represented using a Beta posterior distribution and evaluated using a conservative lower posterior bound rather than the posterior mean. Evidence is maintained separately for each action type and machine class so that successful outcomes in one context do not automatically transfer to another. The project uses a public system-log dataset to inform incident-type distributions and a deterministic simulator to evaluate the gate before any real infrastructure action is considered. The current implementation provides the core decision gate, configurable action and threshold parameters, Bayesian evidence tracking, and safety tests. Subsequent stages will evaluate outcome-based learning, benchmark the gate against simpler baselines, and measure automatic resolution, recovery time, human interruptions, and unsafe automatic executions.**

***Index Terms*—Automated remediation, self-healing infrastructure, Bayesian evidence, trust gate, infrastructure reliability**

I. Introduction

Modern infrastructure systems increasingly use automation to detect failures and initiate remediation actions. Automation can reduce recovery time and decrease the operational workload placed on human engineers. However, an automation system that executes every proposed action without considering context can turn an uncertain diagnosis into an unsafe infrastructure change.

The central problem addressed by this project is therefore not failure detection itself, but the decision of whether an already proposed remediation action has earned sufficient trust to be executed automatically.

The project prioritizes limiting unsafe automatic executions rather than maximizing the number of actions executed automatically. The current implementation establishes the trust gate, configurable action definitions, Bayesian evidence tracking, and safety constraints using a deterministic simulator. Later stages of the project will evaluate how accumulated evidence influences the transition between ESCALATE, PROPOSE, and AUTO decisions.

- **AUTO**: the action may be executed automatically.
- **PROPOSE**: the action may be proposed to a human operator for approval.
- **ESCALATE**: the incident should remain under human control.

An action can initially require human involvement and later become eligible for automatic execution as successful evidence accumulates. Conversely, failures can reduce the accumulated evidence and prevent continued automatic execution.

The design is intentionally based on three factors. Reversibility represents how easily an action can be undone. Blast radius represents the potential scope of impact if an action is incorrect. Accumulated evidence represents historical observations of whether a particular action has successfully resolved incidents in a particular machine-class context.

The project prioritizes limiting unsafe automatic executions rather than maximizing the number of actions executed automatically. The final evaluation will therefore examine whether the trust gate provides useful automation while maintaining this safety objective.

II. Problem Statement

A conventional automated remediation workflow can be represented as:

$$\text{Failure Detection} \rightarrow \text{Candidate Action} \rightarrow \text{Action Execution} \tag{1}$$

$$\text{Failure Detection} \rightarrow \text{Candidate Action} \rightarrow \text{EarnedAutonomy Gate} \tag{2}$$

$$\text{EarnedAutonomy Gate} \rightarrow \begin{cases} \text{AUTO} \\ \text{PROPOSE} \\ \text{ESCALATE} \end{cases} \tag{3}$$

The missing component is a trust decision between the candidate action and its execution.

An action that is appropriate in one context may not be appropriate in another. For example, restarting a stateless service may be relatively reversible, while draining a compute node may affect a larger amount of active work. Similarly, an action that has repeatedly succeeded on one machine class should not automatically inherit the same confidence when applied to a different machine class.

TABLE I
BGL DATASET SUMMARY

Property	Value
Total records	4,747,963
Normal records	4,399,503
Alert records	348,460
Alert percentage	7.34%
Distinct labels	42

EarnedAutonomy addresses this problem by evaluating a candidate action using reversibility, blast radius, and accumulated evidence before determining the level of autonomy permitted.

III. PROJECT SCOPE

The project is deliberately limited to the decision layer governing remediation actions.

EarnedAutonomy does not:

- detect infrastructure failures;
- execute remediation actions on real infrastructure;
- build an infrastructure monitoring dashboard;
- use machine learning to detect failures;
- replace an existing remediation executor;
- introduce additional trust factors beyond the specified design.

The intended architecture is:

$$\begin{aligned}\text{Fault Detector} &\rightarrow \text{Candidate Action} \\ &\rightarrow \text{EarnedAutonomy Gate} \rightarrow \text{Decision}\end{aligned} \tag{4}$$

The trust gate therefore acts as a control point between detecting a problem and permitting an automated response.

IV. DATASET ANALYSIS

A. Public Log Dataset

The project uses the Blue Gene/L (BGL) system log dataset as a public source for deriving the distribution of incident types used by the simulator. The dataset is used to inform the shape of the simulated incident stream rather than to train a failure-detection model.

The analyzed dataset contains 4,747,963 log records. Of these, 4,399,503 are normal records and 348,460 are alert records, corresponding to an alert rate of approximately 7.34%. The dataset contains 42 distinct alert labels.

The most frequent observed labels include KERNDTLB, KERNSTOR, APPSEV, KERNMNTF, and KERNTERM. These observations were used to inform the simplified incident taxonomy and incident-type weights used by the simulator.

B. Incident Taxonomy

The simulator currently represents five broad incident types:

- kernel error;
- storage error;
- filesystem failure;
- communication failure;
- application error.

The mapping is intentionally broader than the original log labels. The purpose is to create a small and interpretable incident space for evaluating the trust gate rather than reproducing the complete BGL logging system.

V. INCIDENT AND ACTION MODELING

A. Incident Model

Each simulated incident contains:

- an incident identifier;
- an incident type;
- a machine class;
- a timestamp.

The simulator currently uses three machine classes:

- `compute_node`;
- `storage_node`;
- `scheduler`.

The current simulator generates 1,000 incidents for the Week 1 and Week 2 verification workflow.

B. Action Types

The project defines six remediation action types. The action descriptions follow the project guide, while the reversibility and blast-radius values are the current configuration values used by the implementation.

TABLE II
REMEDIATION ACTIONS AND INITIAL FACTORS

Action	Reversibility	Blast Radius
Restart stateless service	0.9	0.1
Requeue failed job	0.9	0.1
Reset device driver	0.6	0.3
Drain node and migrate work	0.5	0.7
Reclaim disk space	0.3	0.5
Escalate to human	1.0	0.0

The values are normalized to the range $[0, 1]$. Reversibility is higher when an action can be easily undone. Blast radius is higher when an incorrect action can affect a larger portion of the system.

VI. SIMULATOR

The simulator generates incidents according to the simplified incident taxonomy. The current incident-type probabilities are:

These weights are modeling assumptions informed by the observed BGL label distribution. They are not intended to represent an exact mapping from BGL labels to infrastructure failure causes.

TABLE III
SIMULATOR INCIDENT-TYPE WEIGHTS

Incident Type	Weight
Kernel error	0.5235
Storage error	0.1831
Filesystem failure	0.0930
Communication failure	0.0177
Application error	0.1827

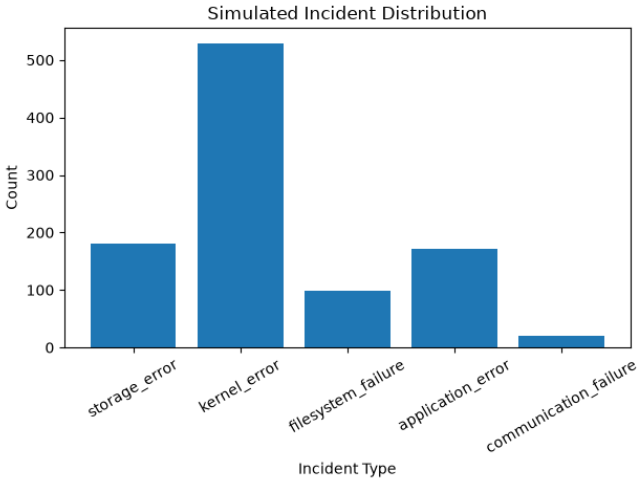


Fig. 1. Simulated incident distribution for 1,000 generated incidents.

Figure 1 shows the distribution produced by the simulator for the Week 1 1,000-incident run.

The simulator currently does not model remediation outcomes. Outcome modeling is introduced in a later project stage so that the evidence component can learn from successful and unsuccessful actions.

VII. EARNED AUTONOMY TRUST GATE

A. Three Factors

The trust gate combines three factors.

1) Reversibility: Reversibility represents how easily a remediation action can be undone. Actions with higher reversibility are treated as safer candidates for automation.

2) Blast Radius: Blast radius represents the potential scope of impact if an action is incorrect. A larger blast radius increases the risk associated with automatic execution.

3) Evidence: Evidence represents the accumulated history of whether an action has successfully resolved previous incidents.

The evidence is maintained separately for each combination of:

$$(\text{action type}, \text{machine class}) \tag{5}$$

This context separation prevents successful outcomes from one machine class from automatically becoming evidence for another machine class.

B. Bayesian Evidence Model

Evidence is represented using a Beta posterior distribution. Each action–machine-class combination starts with a uniform prior:

$$p \sim \text{Beta}(1, 1) \tag{6}$$

A successful remediation increments the α parameter:

$$\alpha \leftarrow \alpha + 1 \tag{7}$$

A failed remediation increments the β parameter:

$$\beta \leftarrow \beta + 1 \tag{8}$$

The posterior mean is:

$$E[p] = \frac{\alpha}{\alpha + \beta} \tag{9}$$

However, the trust gate does not use the posterior mean as its evidence score. Instead, it uses a conservative lower posterior percentile. The current implementation uses the 5th percentile of the Beta posterior:

$$E_{LB} = F^{-1}_{\text{Beta}(\alpha,\beta)}(0.05) \tag{10}$$

This conservative estimate makes the gate less willing to grant autonomy when the evidence is limited or uncertain. The implementation uses SciPy's Beta distribution inverse cumulative distribution function to calculate this value directly.

C. Trust and Risk

The trust score combines reversibility with the conservative evidence estimate:

$$\text{trust} = \text{reversibility} \times E_{LB} \tag{11}$$

The risk score is the normalized blast radius:

$$\text{risk} = \text{blast radius} \tag{12}$$

The current implementation uses the following configurable thresholds:

$$\text{AUTO_THRESHOLD} = 0.65 \tag{13}$$

$$\text{PROPOSE_THRESHOLD} = 0.35 \tag{14}$$

$$\text{RISK_CEILING} = 0.60 \tag{15}$$

These values are engineering configuration choices for the current implementation. A later sensitivity analysis will examine how threshold choices affect the benchmark results.

D. Decision Logic

The current gate applies the following decision rule:

$$\begin{cases} \text{AUTO}, & h \wedge (\text{trust} > T_A) \wedge (\text{risk} < R_C) \\ \text{PROPOSE}, & \text{trust} > T_P \\ \text{ESCALATE}, & \text{otherwise} \end{cases} \tag{16}$$

where h indicates whether the action–machine-class combination has previously observed an outcome.

The history requirement ensures that an action with no observed outcomes cannot be automatically executed. The risk ceiling provides an additional constraint on automatic execution, so an action cannot become AUTO solely by accumulating successful outcomes when its blast radius is too high.

This produces three possible autonomy levels: AUTO permits automatic execution, PROPOSE requests human approval, and ESCALATE requires human handling.

VIII. Implementation

The core implementation is organized into small components:

- `src/gate.py` implements the trust gate and decision logic.
- `src/evidence.py` implements Beta evidence tracking and lower-bound calculation.
- `src/simulator.py` generates simulated incidents.
- `config/actions.yaml` stores action reversibility and blast-radius values.
- `config/thresholds.yaml` stores the decision thresholds.

The implementation uses Python together with NumPy, pandas, SciPy, PyYAML, and matplotlib. The configuration values are kept outside the core decision code so that action factors and thresholds can be changed without modifying the gate logic.

IX. Week 2 Verification

The Week 2 implementation was verified using automated tests covering the required safety properties of the trust gate. The test suite contains five tests and all five passed.

The tests verify that:

- an action with no history never returns AUTO;
- a high-blast-radius action does not return AUTO;
- moderate evidence can produce a PROPOSE decision;
- sufficiently strong evidence for a low-risk action can produce AUTO;
- insufficient evidence produces ESCALATE.

The implementation was also run end-to-end on 1,000 simulated incidents. Because the Week 2 simulator does not yet provide remediation outcomes, no action–machine-class combination has accumulated evidence. As a result, all 1,000 decisions were ESCALATE.

Table **??** summarizes the observed decision counts.

X. Project Progress and Next Stage

The current implementation establishes the first complete version of the trust gate and its safety constraints. The core decision logic, Bayesian evidence tracking, configurable action factors and thresholds, simulator, and automated tests are now implemented.

The next stage introduces outcome modeling. The simulator will distinguish between actions that correctly resolve an incident, actions that appear to work but allow the underlying cause to return, and harmful actions that worsen the incident. These outcomes will update the Beta evidence associated with each action and machine-class context.

The project will then evaluate whether an action can move from ESCALATE to PROPOSE and eventually to AUTO as evidence accumulates. The evaluation will also examine whether failures can reduce accumulated trust and prevent unsafe automatic execution.

Later stages will compare EarnedAutonomy with simpler always-ask and always-act baselines. The evaluation will use automatic resolution rate, mean time to recovery, human interruptions, and unsafe automatic executions. A deterministic random seed will be used so that benchmark runs can be reproduced.

XI. Limitations and Threats to Validity

The current implementation has several limitations.

First, the project currently uses a simulator rather than a production infrastructure environment. Therefore, simulated outcomes cannot fully reproduce the complexity of real remediation scenarios.

Second, the machine classes are simplified into three broad categories: compute nodes, storage nodes, and schedulers. Real infrastructure may require a more detailed contextual model.

Third, the current reversibility and blast-radius values are configuration assumptions rather than measurements from production observations. These values may therefore differ from those of real remediation systems.

Fourth, the BGL dataset is used to inform the shape of the simulated incident distribution rather than to represent a direct production environment. The simplified incident taxonomy and weighting therefore do not reproduce the complete characteristics of the original dataset.

Finally, the current Week 2 implementation does not yet model remediation outcomes. Consequently, the present results demonstrate the behavior of the initial trust gate and its safety constraints, but do not yet establish the effectiveness of learned evidence or the final benchmark performance.

XII. Conclusion and Future Work

EarnedAutonomy introduces a trust decision layer for automated infrastructure remediation. The current implementation combines reversibility, blast radius, and context-specific Bayesian evidence to determine whether a candidate action should be automatically executed, proposed for human approval, or escalated to a human operator.

The Week 2 implementation establishes the core trust gate, Beta evidence tracking, configurable action and threshold parameters, simulator, and automated safety tests. The current verification confirms the required safety behavior, while the later project stages will evaluate outcome-based learning and benchmark performance.

Future work will extend the simulator with remediation outcomes, evaluate changes in autonomy as evidence accumulates, and compare EarnedAutonomy with always-ask and always-act baselines. The final evaluation will focus on automatic resolution rate, mean time to recovery, human interruptions, and unsafe automatic executions, with particular attention to limiting unsafe automatic actions.

REFERENCES

[1] A. Oliner and J. Stearley, "What Supercomputers Say: A Study of Five System Logs," in *Proc. IEEE International Conference on Dependable Systems and Networks*, 2007.

[2] A. Oliner, "Blue Gene/L Supercomputer Logs," LogHub / Zenodo, dataset record 8196385.

[3] J. O. Kephart and D. M. Chess, "The Vision of Autonomic Computing," *Computer*, vol. 36, no. 1, pp. 41–50, 2003.

[4] M. C. Huebscher and J. A. McCann, "A Survey of Autonomic Computing—Degrees, Models, and Applications," *ACM Computing Surveys*, vol. 40, no. 3, 2008.

[5] A. Gelman, J. B. Carlin, H. S. Stern, D. B. Dunson, A. Vehtari, and D. B. Rubin, *Bayesian Data Analysis*, 3rd ed. Boca Raton, FL, USA: CRC Press, 2013.